

Text Mining Final Project Proposal: Detecting Fake Job Postings

Chun-Wen Liou

University of Illinois Urbana-Champaign
cwliou2@illinois.edu

Abstract

Fraudulent job postings pose serious risks such as financial scams and identity theft. This project applies text mining techniques to classify job postings as real or fraudulent. Using a Kaggle dataset of 18,000 job ads, we apply pre-processing steps including feature engineering and missing value imputation. Our experiments involve traditional machine learning models, deep learning architectures, and BERT-based transformers. We evaluate model performance using macro-averaged F1-score and balanced accuracy to address the class imbalance in the dataset. Results show that deep learning models achieve the highest performance, but simpler models also perform well with appropriate preprocessing.

1 Introduction and Goal

The proliferation and advancement of online job platforms have made it easier for job seekers to find opportunities. However, this convenience has also given rise to a significant and growing problem: fraudulent job postings. These fraudulent job postings are becoming more and more common, leading to financial scams, identity theft, and wasted time for applicants.

This project aims to address this issue using text mining techniques to detect fraudulent job postings. By analyzing linguistic and structural patterns in job descriptions, company profiles, and other textual features, we seek to develop a classification model capable of distinguishing between real and fraudulent postings. Our goal is to build a robust model that achieves high performance under imbalanced data conditions.

2 Literature review

Detecting fraudulent job postings is a challenging problem that requires careful data handling, feature extraction, and classification techniques. Previous research has explored various machine learning

and deep learning approaches to improve detection accuracy and mitigate class imbalance.

2.1 Machine Learning Approaches

Early approaches to fraudulent job post detection predominantly relied on traditional machine learning models. [Vidros et al. \(2017\)](#) evaluated classifiers such as Random Forest, Naïve Bayes, and Logistic Regression, finding that Random Forest achieved the highest accuracy (89.5%), highlighting its effectiveness for structured fraud detection. Similarly, [Amaar et al. \(2022\)](#) explored feature extraction techniques like TF-IDF and Bag-of-Words, demonstrating that textual data alone provides strong signals for identifying fraudulent activity. These findings support our approach of analyzing linguistic cues from job descriptions, company profiles, and qualifications to distinguish between real and fraudulent postings.

2.2 Advancements with Deep Learning

Recent research has investigated the potential of deep learning models in this domain to capture complex semantic patterns. [Habiba et al. \(2021\)](#) compared multiple data mining techniques, including K-Nearest Neighbors (KNN), Decision Trees (DT), and Deep Neural Networks (DNN). Their results indicated that DNN outperformed traditional ML models.

Building on this, [Akram et al. \(2024\)](#) applied transformer-based models such as BERT and RoBERTa to detect fraudulent job postings, demonstrating that these architectures can capture nuanced linguistic cues more effectively than traditional classifiers.

2.3 Challenges in Class Imbalance

A major challenge in fraudulent job post detection is the extreme class imbalance—fraudulent postings constitute only a small fraction of available data. [He and Garcia \(2009\)](#) emphasize the

adverse effects of class imbalance on model performance, advocating for techniques like SMOTE and cost-sensitive learning. This insight informs our methodology, as addressing class imbalance is critical for improving recall and ensuring that fraudulent job postings are effectively identified.

2.4 Summary of Key Insights

These studies highlight the evolution from traditional machine learning approaches to more advanced deep learning techniques for fraud detection. Based on these insights, our projects implements a range of models—from Naïve Bayes to BERT—with particular emphasis on macro-averaged F1 score and balanced accuracy as evaluation metrics to ensure fair and effective performance under class imbalance.

3 Descriptive statistics of data

This project will utilize the "Real or Fake Job Posting Prediction" dataset from Kaggle, created by Shivam Bansal.¹ The dataset contains 17,880 job postings with 18 columns, including key textual and structured features such as title, location, company_profile, description, requirements, benefits, salary_range, and more, with a binary label that indicates whether the job posting is fraudulent (fraudulent = 1) or real (fraudulent = 0). The fraudulent type distribution of the dataset is shown in Table 1, with 95.16% real postings and only 4.84% fraudulent postings, making it imbalanced.

fraudulent	count	percentage
0(Real)	17014	95.16%
1(Fake)	866	4.84%

Table 1: The distribution fraudulent of in the dataset.

Among these, key features will be used for text classification include job title, location, company profile, job description, and required qualifications since it is expected that these elements contain critical information for identifying fraudulent job postings.

3.1 Example Job Postings

To illustrate the nature of the dataset, we present one example each from a real and a fraudulent job posting.

¹<https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction/data#>

Real Posting Example (Web Developer):

Title: Web Developer, HTML | CSS | JavaScript

Description: PINT is seeking an associate web developer with strong knowledge of HTML, CSS, and JavaScript to help us maintain and build client websites. The role involves front-end development, working with designers and other developers in a fast-paced environment.

Fraudulent Posting Example (Customer Operations Manager):

Title: CUSTOMER OPERATION MANAGER

Description: Customer Care Operations Manager is responsible for supporting our clients' service delivery and driving client satisfaction. This role requires remote availability and offers flexible hours. Basic computer skills and a positive attitude are sufficient to start. Training provided.

These examples highlight the subtle linguistic and structural differences between real and fraudulent postings. While the real posting emphasizes technical requirements and team collaboration, the fraudulent one uses vague terms and promises low-barrier entry without clear qualifications.

4 Preprocessing

4.1 Handling Missing Values

Missing values in text columns (e.g., *location*, *company_profile*, *description*, *requirements*, *benefits*) were filled with the placeholder 'NA'. This ensures that the text data is consistent and can be processed by the model. This approach strikes a balance between simplicity and effectiveness, allowing us to proceed with feature extraction and model development without introducing unnecessary complexity. However, future work could explore more sophisticated methods for handling missing text data, potentially leading to further improvements in model performance.

4.2 Text Preprocessing

For traditional machine learning models such as Naive Bayes, Logistic Regression, Support Vector

Machines, and Random Forest, we applied a standardized text preprocessing pipeline. The text was converted to lowercase, and punctuation, numbers, and stopwords were removed. Lemmatization was then applied to reduce inflected forms to their base versions, improving vocabulary consistency and reducing sparsity.

For deep learning models such as CNN, LSTM, and BiLSTM, the cleaned text was tokenized, converting each document into a sequence of integers. These sequences were then padded to a fixed length. These models operated on the same simplified text as traditional models but processed it as sequences rather than vectorized features.

In contrast, the BERT model required minimal preprocessing. To align with the expectations of its WordPiece tokenizer, we preserved the original casing, punctuation, and token forms. Each input sequence was truncated or padded to a fixed length of 512 tokens. This setup allowed BERT to leverage subword information and capture deeper contextual relationships.

By adapting the preprocessing strategy to the specific needs of each model architecture, we ensured compatibility and performance optimization across all experimental conditions.

5 Feature Extraction and Selection

To prepare the cleaned textual data for model training, we explored three primary feature extraction strategies: Term Frequency–Inverse Document Frequency (TF-IDF), CountVectorizer, and pretrained GloVe embeddings. These methods differ in how they represent textual information numerically, ranging from sparse term-frequency counts to dense vector averages that capture semantic similarity.

Before feature extraction, we performed an 80/20 stratified split of the dataset into training and test sets to ensure balanced class distribution. All vectorizers were fitted exclusively on the training data to prevent data leakage and ensure fair evaluation.

The first method, TF-IDF, assigns weights to words based on their importance in a specific document relative to their frequency in the entire corpus. We limited the vocabulary to the top 2,000 terms in the training set to reduce dimensionality and noise. These TF-IDF features were used to train all traditional machine learning models, including Naive Bayes, Logistic Regression, Support Vector

Machines (SVM), and Random Forest.

As a baseline, we also employed CountVectorizer, which encodes raw word frequencies without considering inverse document frequency. Using the same 2,000-word vocabulary, CountVectorizer features were evaluated across the same set of classifiers to measure the impact of term weighting on model performance.

To incorporate semantic information, we experimented with pretrained GloVe embeddings. Each job posting was represented by averaging the 100-dimensional GloVe vectors of all tokens in the cleaned text. These dense, fixed-length vectors were used as input to a Logistic Regression model.

This diverse set of feature representations enabled us to compare frequency-based and semantics-based text encodings. Our objective was to evaluate not only which vectorization strategy performs best in detecting fraudulent job postings, but also to explore the tradeoffs between interpretability, sparsity, and semantic richness.

6 Modeling Approach

To detect fraudulent job postings using text data, we implemented a wide range of classification models, ranging from traditional algorithms to deep learning and transformer-based architectures. These models were trained on features derived from three types of textual representations—TF-IDF, CountVectorizer, and pretrained GloVe embeddings—each of which captures different aspects of the input text.

We began by establishing performance baselines using four widely adopted machine learning classifiers: Naive Bayes, Logistic Regression, Support Vector Machine (SVM), and Random Forest. These models were evaluated under imbalanced class conditions to understand their sensitivity to minority (fraudulent) cases. We then applied Synthetic Minority Over-sampling Technique (SMOTE) to investigate whether resampling could improve recall and overall macro F1-scores.

Building on these results, we extended our experiments to include deep learning models such as CNN, LSTM, and BiLSTM, which are capable of capturing sequential and contextual patterns in job descriptions. Finally, we fine-tuned a pretrained BERT transformer model, which leverages subword tokenization and contextual embeddings to model language with greater nuance.

In addition to model training and evaluation, we

also analyzed feature importance based on the coefficients of a Logistic Regression classifier trained with TF-IDF features. This analysis helped us interpret which words were most indicative of real versus fraudulent postings.

Due to the significant class imbalance in the dataset, we adopted macro-averaged precision, recall, and F1-score as our primary evaluation metrics, along with balanced accuracy as a secondary metric. These metrics ensure that both classes are equally weighted during evaluation, regardless of the underlying label distribution. All subsequent model comparisons are reported using these metrics.

6.1 Baseline Models without Resampling

We first established baseline results using four traditional machine learning models—Multinomial Naive Bayes, Logistic Regression, Support Vector Machine (SVM), and Random Forest—without applying any resampling techniques. These models were trained using two types of feature representations: TF-IDF and CountVectorizer. The goal was to evaluate their ability to detect fraudulent job postings in an imbalanced setting. Table 2 summarizes the performance of each model across the two vectorization techniques using macro-averaged metrics.

Among the baseline combinations, Logistic Regression using CountVectorizer achieved the best macro F1-score of 0.90 and a balanced accuracy of 0.884. SVM with TF-IDF also performed strongly, with an F1-score of 0.88. Naive Bayes models achieved high recall but suffered from lower precision, especially when paired with CountVectorizer. These results show that even without resampling, traditional models can achieve reasonable performance with appropriate feature engineering.

In the following subsection, we explore an alternative dense feature representation, GloVe word embeddings, used in conjunction with Logistic Regression to assess the value of distributed semantic information.

6.1.1 GloVe Embeddings with Logistic Regression

To incorporate distributed semantic representations into our baseline models, we experimented with pretrained GloVe embeddings. Specifically, we used 100-dimensional GloVe vectors to represent each word in a job posting. A document vector was constructed by averaging all token embeddings in

the cleaned text.

We then trained a Logistic Regression classifier on these dense vectors. This method provides a semantically informed feature space that captures contextual similarity between words, unlike the sparse and local nature of TF-IDF or CountVectorizer.

The model achieved a balanced accuracy of 0.779 and a macro F1-score of 0.85, comparable to Random Forest with TF-IDF features. One possible explanation is that averaging embeddings tends to dilute key lexical signals, making it harder for linear classifiers to distinguish between subtle patterns present in scam postings.

Nevertheless, this experiment demonstrates that GloVe embeddings are a viable alternative to sparse representations and it did not outperform the best sparse-vector baselines.

6.2 Resampling with SMOTE

Due to the significant class imbalance in our dataset—where fraudulent postings comprise less than 5% of all records—we applied the Synthetic Minority Over-sampling Technique (SMOTE) to augment the minority class during training. Rather than simply duplicating existing samples, SMOTE generates synthetic examples by interpolating between nearby instances in the feature space, thereby promoting better generalization and reducing the risk of overfitting.

The same four traditional classifiers—Naive Bayes, Logistic Regression, Support Vector Machines (SVM), and Random Forest—were re-trained on the SMOTE-resampled datasets using both TF-IDF and CountVectorizer features. We evaluated their performance using macro-averaged balanced accuracy, precision, recall, and F1-score. The results are summarized in Table 3.

In most cases, applying SMOTE notably improved recall and balanced accuracy. For instance, Logistic Regression with TF-IDF saw its recall increase from 0.69 to 0.86 and balanced accuracy from 0.693 to 0.839. Naive Bayes with the same feature set showed even more dramatic gains in recall—from 0.60 to 0.84—while maintaining high precision. These results demonstrate that SMOTE effectively enhanced the models' sensitivity to fraudulent job postings, particularly when used with TF-IDF.

However, the effect of SMOTE was not universally positive. For some model-feature pairings,

Vectorizer	Model	Balanced Acc.	Precision	Recall	F1-Score
TF-IDF	Naive Bayes	0.602	0.98	0.60	0.66
TF-IDF	Logistic Regression	0.693	0.98	0.69	0.77
TF-IDF	SVM	0.812	0.99	0.81	0.88
TF-IDF	Random Forest	0.779	0.99	0.78	0.85
CountVec	Naive Bayes	0.875	0.65	0.88	0.69
CountVec	Logistic Regression	0.884	0.93	0.88	0.90
CountVec	SVM	0.702	0.98	0.70	0.78
CountVec	Random Forest	0.779	0.99	0.78	0.85

Table 2: Performance of baseline models using TF-IDF and CountVectorizer (macro average).

Vectorizer	Model	Balanced Acc.	Precision	Recall	F1-Score
TF-IDF	Naive Bayes + SMOTE	0.839	0.84	0.84	0.84
TF-IDF	Logistic Regression + SMOTE	0.839	0.86	0.86	0.86
TF-IDF	SVM + SMOTE	0.802	0.84	0.80	0.82
TF-IDF	Random Forest + SMOTE	0.836	0.86	0.84	0.85
CountVec	Naive Bayes + SMOTE	0.875	0.65	0.88	0.69
CountVec	Logistic Regression + SMOTE	0.851	0.86	0.85	0.85
CountVec	SVM + SMOTE	0.822	0.87	0.82	0.84
CountVec	Random Forest + SMOTE	0.833	0.85	0.83	0.84

Table 3: Model performance after applying SMOTE (macro average metrics).

such as Naive Bayes with CountVectorizer, performance gains were limited or inconsistent, particularly in terms of precision.

Overall, our experiments demonstrate that the benefits of SMOTE are not universal. While it significantly enhanced macro F1-scores for certain model-feature combinations—such as Logistic Regression with TF-IDF—it led to marginal or even negative impacts for others. These findings underscore the importance of evaluating resampling techniques in the context of both classifier architecture and feature representation, rather than assuming uniform improvements across models.

6.3 Deep Learning Method

To explore the capability of sequential deep learning models in detecting fraudulent job postings, we implemented three neural architectures using Keras: a Convolutional Neural Network (CNN), a Long Short-Term Memory (LSTM) network, and a Bidirectional LSTM (BiLSTM).

Table 4 summarizes the performance of all three models. Both CNN and BiLSTM achieved strong macro F1-scores of 0.96, with balanced accuracies of 0.93. These models correctly identified most fraudulent postings with high precision, suggesting that both architectures can effectively detect linguistic patterns indicative of scam jobs. In contrast,

the LSTM model failed to detect any fraudulent postings, resulting in a macro F1-score of 0.49 and a balanced accuracy of only 0.50. This suggests a strong bias toward the majority class and limited contextual awareness due to its unidirectional structure.

While CNN and BiLSTM performed similarly, both models were able to capture linguistic features indicative of fraudulent content, leading to high detection performance. We also conducted qualitative error analysis and found that misclassified examples were often very short, vague, or highly polished job postings that closely resembled legitimate listings, making them difficult to detect—even for human readers.

6.4 Experimenting with BERT

To explore another deep learning approaches in detecting fraudulent job postings, we fine-tuned a pre-trained BERT-base model using the HuggingFace Transformers library. Instead of preprocessing, we preserved the natural structure of the job postings by retaining casing, punctuation, and numbers.

The text was tokenized using BERT’s WordPiece tokenizer with a maximum sequence length of 512. The dataset was split with stratification to maintain class distribution, and the model was fine-tuned for five epochs using the AdamW optimizer with a

Model	Balanced Acc.	Precision	Recall	F1-Score
CNN	0.93	0.99	0.93	0.96
LSTM	0.50	0.49	0.50	0.49
BiLSTM	0.93	0.99	0.93	0.96

Table 4: Performance of deep learning models on fraudulent job detection (macro average).

linear learning rate scheduler and warmup steps.

After training, the model achieved a balanced accuracy of 0.93 and a macro-averaged F1-score of 0.94 on the test set, indicating strong performance in distinguishing between real and fraudulent postings. Compared to other deep learning models (e.g., CNN and BiLSTM), BERT performed competitively.

In summary, BERT demonstrated competitive performance and has the potential to further improve fraud detection when given sufficient computational resources and hyperparameter tuning.

7 Feature Importance Analysis

To gain interpretability from the model’s decision-making process, we analyzed the learned coefficients of a Logistic Regression classifier trained on TF-IDF features. Because Logistic Regression is a linear model, each feature receives a weight that reflects its contribution to the classification decision. Positive coefficients are associated with a higher likelihood of the posting being fraudulent, while negative coefficients indicate stronger association with real job postings.

We extracted the top 20 features with the largest positive coefficients (indicative of fake postings) and the top 20 with the most negative coefficients (indicative of real postings). Table 5 presents a sample of these highly weighted words.

Fraud-indicative terms included words such as “immediately,” “income,” “benefits,” “limited,” and “position,” which often appear in scam-like language that emphasizes urgency or vague compensation. Conversely, terms such as “requirements,” “team,” “skills,” and “experience” were strongly associated with legitimate postings, suggesting a focus on specific qualifications and work environment.

These insights provide human-understandable explanations for the model’s predictions and offer qualitative support for the patterns learned by the classifier. In practical settings, such interpretability can assist domain experts or platform moderators in manually reviewing suspicious postings.

8 Error Analysis and Interpretation

To better understand the limitations of our models and the nature of challenging cases, we conducted a detailed error analysis across traditional, deep learning, and transformer-based classifiers. This analysis focused on both false negatives (fraudulent postings misclassified as real) and false positives (real postings flagged as fraud), using qualitative examples and performance comparisons to extract insights.

8.1 Errors in Traditional Models

Among traditional classifiers, Logistic Regression with TF-IDF and SMOTE performed well, but struggled with postings that used vague or highly polished language. For example:

"Customer Service Representative — Since 2005, Allied Secured Benefits has helped thousands of clients secure their financial futures..."

Such descriptions mimic authentic corporate tone and structure, which frequency-based features may fail to capture.

Logistic Regression with CountVectorizer also showed strong performance. However, it misclassified some real postings with overly enthusiastic phrasing as fraudulent:

"Join our fast-growing team in a fun and energetic environment!"

These cases suggest that while SMOTE improves recall, it may also increase false positives when lexical features overlap between legitimate and scam postings.

8.2 Patterns in Deep Learning and Transformer Models

Both CNN and BiLSTM achieved high macro F1-scores, but their errors shared two main traits: generic job descriptions and scam postings with well-crafted, professional language. For instance:

Fraud-Indicative Words	Real-Indicative Words
below, american, aptitude, motivated, oriented eg, short, story, send, leveraging balance, break, networking, attention, scalable immediate, excellence, guide, achieve, maximum	digital, handling, distribution, revenue, fun disability, fit, component, university, exceptional hardware, selling, valor, requires, custom findingrecruiting, organizational, english, near

Table 5: Top 20 most indicative words for each class based on Logistic Regression coefficients (TF-IDF).

"With decades of experience and over 5000 experts in 51 countries, MSX International..."

Such listings are difficult to distinguish even for human readers.

BERT also made errors. False negatives often involved scam signals buried deep in long text, while false positives stemmed from emotionally persuasive or marketing-heavy language, such as:

"We are revolutionizing the job market — join the movement and grow with us!"

8.3 Summary of Common Error Patterns

Across all models, the following patterns frequently occurred in misclassified examples:

- **Short or generic descriptions** with too short and little content
- **Polished, professional tone** mimicking legitimate companies
- **Lack of scam-related keywords** such as "bonus" or "startup fee"
- **Exaggerated or promotional language** in real postings

We also found that some misclassified entries lacked fields like `company_profile` or `requirements`, indicating that structural cues and metadata could enhance performance.

These insights suggest that future work could incorporate metadata features, model structural patterns (e.g., missing fields, text length), or expand training data to mitigate dataset-specific biases.

9 Model Comparison and Discussion

To comprehensively evaluate the effectiveness of different modeling strategies, we compared all trained models—including traditional classifiers, deep learning architectures, and a transformer-based BERT model—based on their ability to detect fraudulent job postings. The goal was not only

to identify the best-performing model but also to understand how different combinations of architecture, feature representation, and class balancing affect overall classification performance. All evaluations were conducted using macro-averaged precision, recall, F1-score, and balanced accuracy to ensure fair comparisons under imbalanced class conditions.

Among all models, both CNN and BiLSTM achieved the highest performance, with macro F1-scores of 0.96 and balanced accuracy of 0.93. These models are particularly effective in learning local and sequential text patterns, respectively. CNN leverages convolutional filters to detect important n-gram patterns that are indicative of scam language, while BiLSTM captures context in both forward and backward directions, allowing it to identify inconsistencies or linguistic red flags embedded throughout a posting. These characteristics enable the models to generalize well across varied types of fraud attempts.

BERT also performed competitively ($F1 = 0.94$), confirming the advantage of contextualized subword embeddings. While BERT has the ability to capture subtle semantic nuances, our results show that its improvements over CNN and BiLSTM were marginal given the significantly higher training cost and resource requirements.

Logistic Regression with CountVectorizer (without SMOTE) achieved the best result among traditional classifiers ($F1 = 0.90$, balanced accuracy = 0.884). This result illustrates that well-tuned simple models can still be remarkably effective when paired with strong feature engineering. When combined with SMOTE, Logistic Regression (TF-IDF + SMOTE) reached an F1-score of 0.86, suggesting that oversampling improved recall but slightly reduced precision due to potential oversensitivity to minority patterns. These results highlight that SMOTE is more effective when the model architecture is not inherently robust to class imbalance.

Overall, these findings demonstrate that no single model universally dominates across all eval-

Model	Type	Balanced Acc.	Precision	Recall	F1-Score
LogReg (CountVec)	Traditional	0.884	0.93	0.88	0.90
SVM (TF-IDF)	Traditional	0.812	0.99	0.81	0.88
Naive Bayes (TF-IDF + SMOTE)	Traditional + SMOTE	0.839	0.84	0.84	0.84
LogReg (TF-IDF + SMOTE)	Traditional + SMOTE	0.839	0.86	0.86	0.86
Random Forest (TF-IDF + SMOTE)	Traditional + SMOTE	0.836	0.86	0.84	0.85
LogReg (CountVec + SMOTE)	Traditional + SMOTE	0.851	0.86	0.85	0.85
CNN	Deep Learning	0.930	0.99	0.93	0.96
BiLSTM	Deep Learning	0.930	0.99	0.93	0.96
BERT	Transformer	0.930	0.96	0.93	0.94

Table 6: Summary of best-performing models across architecture categories.

uation criteria. While deep learning models outperform traditional methods in terms of raw performance, simpler models remain valuable due to their interpretability, ease of training, and surprising effectiveness when paired with the right features and resampling techniques.

In terms of qualitative insights, misclassified fraud cases across all models often featured vague descriptions, polished language, or an absence of explicit scam cues. These patterns reinforce the challenge of relying solely on linguistic features for deception detection and suggest a need for richer, context-aware modeling.

10 Conclusion and Future Work

This project applied text mining techniques to detect fraudulent job postings using a dataset from Kaggle. We explored a comprehensive range of feature extraction strategies—including TF-IDF, CountVectorizer, and GloVe—and trained multiple classification models, from traditional algorithms to sequential deep neural networks and pretrained transformer-based models. Our aim was not only to achieve high performance but also to assess how different preprocessing and modeling choices influence fraud detection under class imbalance.

Our experiments demonstrate that deep learning models, particularly CNN and BiLSTM, delivered the strongest overall results. Both achieved a macro F1-score of 0.96 and a balanced accuracy of 0.93, confirming their ability to capture syntactic and semantic fraud indicators through dense, learned representations. The BERT model also performed well (F1 = 0.94), highlighting the value of contextualized language modeling. However, it did not yield a significant performance boost over lighter deep learning models, raising questions about the trade-off between complexity and marginal gains.

Among traditional methods, Logistic Regression paired with CountVectorizer achieved a macro F1-score of 0.90, outperforming more complex models such as Random Forest and SVM. When combined with SMOTE, simpler models showed improved recall and balanced accuracy, demonstrating that resampling can still offer meaningful gains in scenarios where contextual modeling is limited.

Through error analysis, we found that the most difficult cases to classify were short, vague, or overly polished postings that mimic legitimate job descriptions. These insights underline the limitations of purely text-based approaches and point toward the need for more holistic representations.

For future work, we propose expanding the model to incorporate structural metadata (e.g., field completeness), behavioral data (e.g., posting frequency or location patterns), and knowledge-based features (e.g., references to known scam companies). Exploring multilingual datasets and training models that generalize across countries or industries may also prove valuable.

In conclusion, this study illustrates the importance of balancing feature engineering, architecture design, and class rebalancing to develop effective fraud detection systems. With continued refinement, such tools can offer critical protection to job seekers and improve the integrity of online hiring platforms.

References

- N. Akram, R. Irfan, A. S. Al-Shamayleh, A. Kousar, A. Qaddos, M. Imran, and A. Akhunzada. 2024. [Online recruitment fraud \(orf\) detection using deep learning approaches](#). *IEEE Access*, 12:109388–109408.
- A. Amaar, W. Aljedaani, F. Rustam, S. Ullah, V. Rupapara, and S. Ludi. 2022. [Detection of fake job](#)

postings by utilizing machine learning and natural language processing approaches. *Neural Processing Letters*, 54(3):2219–2247.

S. U. Habiba, Md. K. Islam, and F. Tasnim. 2021. [A comparative study on fake job post prediction using different data mining techniques](#). pages 543–546.

Haibo He and E. A. Garcia. 2009. [Learning from imbalanced data](#). *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284.

S. Vidros, C. Koliass, G. Kambourakis, and L. Akoglu. 2017. [Automatic detection of online recruitment frauds: Characteristics, methods, and a public dataset](#). *Future Internet*, 9(1):6.